Odenc compiler project 1

Abdihafiid Ahmed Arab Sadiq Tarek Kamel Ali Ibrahim

November 1, 2026

Task 1:

We extended the LangNum grammar by using regular expressions to support binary, decimal and hexadecimal integers. In the regex we made sure that both BINARY_LITERAL and HEX_LITERAL, start with the proper prefix, and including DECIMAL_LITERAL all three support negative value and underscore sbetween digits.

```
BINARY_LITERAL: /-?0b[01]+(_?[01]+)*/
DECIMAL_LITERAL: /-?[0-9]+(_?[0-9]+)*/
HEX_LITERAL: /-?0x[0-9a-fA-F]+(_?[0-9a-fA-F]+)*/
```

The pattern also ensures that underscores cant appear at the beginning or the end, and makes sure the prefixes are case sensitive so 0B and 0X aren't valid. The num_literal transformer removes the underscore and converts each integer according to its appropriate base. Binary with base 2, hex with 16 and decimal with 10. The string is sliced up to 2, since the prefixes are the same everytime, and up to 3 if its a negative too. what matters in the conversion is what comes after the sign and prefix. The implementation was tested by writing 5 examples in progs/num named task1_1→5.odc, with the expected outcome and running the ./check.sh script which they all passed.

Task 2:

We implemented a semantic analysis check for the variables in Odenc, to detetct errors before program execution. We implemented two checks cvd_program() and rbw_program() The cvd_program() method checks that every variable used in an expression has been declared, it collects the declared variable name in a set and goes through the statements in the program to check if it fits, if not it raises an undeclared variable error SemCheckUndeclaredVariable. Likewise for the rbw_program() but this one checks if the variables are read before they have been assigned a value, it keeps a set of initialized variables, and goes through the statements in program order ie For an assignment, it checks the right hand-side expression before marking the target variable as initialized. For a print statement, it checks the expression to ensure that every variable it reads has already been initialized. If a variable is read too early it raises an error SemCheckUninitializedVariable.n We tested our implementation first

by running ./check.sh like we did in task 1, but it only runs the existing compiler and interpreter tests and does not include SemCheck, so we manually checked it by parsing small programs with LangVar().parse() and calling them directly.

```
Var x; x = 1; print(x);
```

passed rbw_program, meanwhile:

```
var x; print(x); and var x; x = x +1;
```

both raised a SemCheckUnintializedVariable: x and likewise for

```
var x; x = y +1;
```

made cvd_program() raise SemCheckUndeclaredVariable: y

Task 3:

In task 3 we implemented a partial evaluation that optimized both operrands of each binary operation, and if both are constants it evaluates the operation and replaces it with a single constants... Therefore nested expressions like 4+3*2 for instance becomes 10 right away making it effeciently optimized. To prevent it from falling back to Python's default operator's we used the interpreter's (to_int64, trunc_div, trunc_mod) to make sure the result follows a 64-bit wrap and trunct so it truncates towards zero instead of negative infinity. We unit tested our implementation using test_partial_eval.py which checks that optimized LangVar programs still meet their expectations including errors, we used claude to write ops_script.py a small program that goes through all the programs in progs/ including five other additional tests from us (task3_1→ 5.odc) and spit out how many operations each program had before and after optimization. Here is a small sample of optimized tests and how much (percent wise) were all the programs optimized.

Program	Before	After	Reduction
int/20_all_operators	9	0	100%
var/13_all_arithmetic_operators	7	4	42.9%
var/task3_1	5	0	100%
var/task3_2	1	1	0%
var/task3_3	3	0	100%
var/task3_4	5	0	100%
var/task3_5 (1/0)	1	1	0%
All programs	227	141	37.9%

Task 4:

In this task we added unary minus to LangVar by extending it into LangUnary. This lets us write things like print(-5);, print(-x); and print(-(2+3)); instead of always subtracting from zero. We kept the LangVar grammar and added an expr_unary rule for the minus sign. We placed it before multiplication and division, so an expression like -2 * 3 is understood as (-2) * 3, just like in C.

The main rule is:

```
?expr_unary: INT64_MIN_LITERAL -> expr_int64_min
           | "-" atom          -> expr_neg
           | atom
```

By using atom after the minus, we made sure print(–1); is not allowed. But print(1 - -1); and print(-(-1)); are allowed. We also adjusted the number tokens so they do not include the minus sign themselves. We added a NegOp node to the AST to store what needs to be negated.

```
case NegOp(operand):
    value = self.interp_expr(operand, ctx)
    assert isinstance(value, int)
    return to_int64(-value)
```

The interpreter calculates that expression first, then changes its sign. For example, print(-(2+3)); gives -5. We also updated the pretty printer for the new expression. To test it, we added 13 programs in progs/unary and 12 unit tests. They passed together with the original unary program. We ran the full check script. The compiler and interpreter tests passed